

```
int a=6,b=3; // We have M-A-S-G-E-A-R-O-Ca-Co in the expression.
a = a/b&b + b||a*b-a^b < <b&&a%b|a; // Let's substitute the values of a and b in Exp.
= 6/3&3 + 3||6*3-6^3 < <3&&6%3|6; // First Division (6/3)
= 2&3 + 3||6*3-6^3 < <3&&6%3|6; // Next Multiplication (6x3)
= 2&3 + 3||18-6^3 < <3&&6%3|6; // Next Modulo division (6%3)
= 2&3 + 3||18-6^3 < <3&&0|6; // Next is Addition (3 + 3)
= 2&6||18-6^3 < <3&&0|6; // Next is Subtraction (18-6)
= 2&6||12^3 < <3&&0|6; // Next is Shift Left (3 < 3)
= 2&6||12^24&&0|6; // Next is bitwise AND
= 2||12^24&&0|6; // Next is bitwise EXOR
= 2||20&&0|6; // Next is Bitwise OR
= 2||20&&6; // Next is Logical AND (20&&6) = 1
= 2||1; // Next is Logical OR (2||1) = 1
= 1.
```

Fig. 6: With multiplicative, additive, bit-wise and compound conditional (&& and ||) operators

```
int a=6,b=3; // We have M-A-S-G-E-A-R-O-Ca-Co-!- in the expression.
a + = a/b > > 1&a + b|b*a-b^a < <b|a%b;
= 6/3 > > 1&6 + 3|3*6-3^6 < <3|6%3; // First 6/3 is performed.
= 2 > > 1&6 + 3|3*6-3^6 < <3|6%3; // Next is (3x6)
= 2 > > 1&6 + 3|18-3^6 < <3|6%3; // Next is (6%3)
= 2 > > 1&6 + 3|18-3^6 < <3|0; // Next is (6 + 3)
= 2 > > 1&9|18-3^6 < <3|0; // Next is (18-3)
= 2 > > 1&9|15^6 < <3|0; // Next is (2 > 1)
= 1&9|15^6 < <3|0; // Next is (6 < 3)
= 1&9|15^48|0; // Next is (1&9)
= 1|15^48|0; // Next is (15^48)
= 1|63|0; // Next is (1|63)
= 63|0; // Next is (63|0)
= 63;
Therefore a = a + 63 = 69; // Finally, (a = a + 3);
```

Fig. 7: C expression with arithmetic, bit-wise (Shift, AND, EXOR, OR) and assignment operators

```
int a=5,b=7; // We have M-A-S-G-E-A-R-O-Ca-Co-!- in the expression.
a < < = a + b^2%4 + b/a-a > b|a > > 1^b:b < <2&b|3;
= 5 + 7^2%4 + 7/5-5 > 7|5 > > 1^7:7 < <2&7|3; // Multiplication of (7x2)
= 5 + 14%4 + 7/5-5 > 7|5 > > 1^7:7 < <2&7|3; // Next is (14%4)
= 5 + 2 + 7/5-5 > 7|5 > > 1^7:7 < <2&7|3; // Next is (7/5)
= 5 + 2 + 1-5 > 7|5 > > 1^7:7 < <2&7|3; // Next is (5 + 2)
= 7 + 1-5 > 7|5 > > 1^7:7 < <2&7|3; // Next is (7 + 1)
= 8-5 > 7|5 > > 1^7:7 < <2&7|3; // Next is (8-5)
= 3 > 7|5 > > 1^7:7 < <2&7|3; // Next is (5 > 1)
= 3 > 7|2^7:7 < <2&7|3; // Next is (7 < 2)
= 3 > 7|2^7:2&&7|3; // Next is (3 > 7) which = 0.
= 0|2^7:2&&7|3; // Next is (2&&7) = 4
= 0|2^7:4|3; // Next is (2^7) = 5
= 0|5:4|3; // Next is (4|3) = 7
= 0|5:7; // Next is Conditional Operator FALSE part = 7
= 7; // Next Left-shift in combination with assignment
a = a < < 7;
= 640
```

Fig. 8: C expression with arithmetic, bit-wise, relational, conditional and assignment operators

additive, bit-wise and assignment operators in a single expression, precedence of operators will come

in force. As per the operator's rhyme from left to right, value can be calculated. **EFY**

Share Your Knowledge

If you are an expert in your field of electronics, why not share your knowledge with the world?



Electronics For You invites electronics professionals and industry experts to write articles on their area of expertise and interest from different perspectives

Some of the topics we want you to write about are:

- Photonics: Optics & Optoelectronics
- Automation
- Power Electronics
- Components
- Telecom

So if you have an interesting topic or original article to share, please let us know at efyeditorial@efy.in

We would love to hear your good ideas too!

electronics
FOR YOU